Hagstofa AI – PxWeb API Backend (Track 2)

This project contains a dynamic **PxWeb API-driven backend** designed for Statistics Iceland (Hagstofa Íslands). Instead of relying on a pre-built static vector database, Track 2 focuses on live querying, automated API request generation, and real-time data retrieval directly from the official PxWeb API endpoints.

🛠️ Requirements & Setup (`requirements.txt`)

Create a file named `requirements.txt` in the root of your repository with the following contents:

```
None
fastapi
uvicorn
requests
pydantic
pandas
```

System Requirements & Preparation:

1. **Python 3.10+** installed on your machine.
2. An active internet connection to communicate live with the Statistics Iceland PxWeb API (`https://px.hagstofa.is/pxis/api/v1/is/`).

📂 GitHub Repository Structure

Your GitHub repository for Track 2 should look like this:

```
None
hagstofa-ai-track2/
│
├── app.py                      # Main FastAPI application & PxWeb API integration
├── requirements.txt            # Python dependencies
├── README.md                   # This documentation file
└── .gitignore                  # Files/folders to ignore in Git
```

⚙️ Step 1: Environment Setup

Open a terminal in your project root directory and run the following commands:

1. **Create a virtual environment:**

```
None
python -m venv venv
```

2. **Activate the virtual environment:**
    - On Mac / Linux:

```
None
source venv/bin/activate
```

    - On Windows (CMD / PowerShell):

```
None
venv\Scripts\activate
```

3. **Install the required packages:**

```
None
pip install --upgrade pip
pip install -r requirements.txt
```

🚀 Step 2: Running the Application

Once your environment is set up and `app.py` is in place, start the FastAPI server:

```
None
uvicorn app:app --reload
```

- The server will run at: `http://127.0.0.1:8000`
- Interactive API Documentation (Swagger UI): `http://127.0.0.1:8000/docs`
- ReDoc Documentation: `http://127.0.0.1:8000/redoc`

🔍 Testing the PxWeb API Endpoint via `curl`

You can test the dynamic query endpoint using `curl` to fetch live data directly through the backend:

```
None
curl -X POST "http://127.0.0.1:8000/query" \
     -H "Content-Type: application/json" \
     -d '{"prompt": "Get population statistics for municipalities in 2024"}'
```

🛑 `.gitignore`

To ensure local binary files, caches, or virtual environments are not tracked by Git, include a `.gitignore` file with:

```
None
venv/
__pycache__/
*.pyc
.env
```